Name: Farid Javed Shaikh

Roll no: B-36

Branch:- AI&DS(B)

Experiment:-07

Aim: Implement and analyze various probability distributions in Python, including Binomial, Poisson, and Normal distributions.

THEORY

1.Difference between Binomial, Poisson, and Normal Distributions

a.Binomial Distribution

Type: Discrete

Parameters: n (number of trials), p (probability of success)

Trials: Fixed number of trials

Outcome: Each trial has two possible outcomes — success or failure

Shape: Symmetric when p = 0.5, otherwise can be skewed

Explanation: The Binomial Distribution models the number of successes in a fixed number of independent trials, where each trial has the same probability of success.

Example: Tossing a coin multiple times and counting the number of heads

b.Poisson Distribution

Type: Discrete

Parameter: λ (lambda — average rate of occurrence)

Trials: Not fixed; events occur over a continuous interval (time, space, etc.)

Outcome: Counts the number of events in a given interval

Shape: Right-skewed when λ is small; becomes more symmetric as λ increases

Explanation: The Poisson Distribution models how many times an event occurs in a fixed interval, assuming events happen independently and at a constant average rate.

Example: Number of phone calls received in an hour

c.Normal Distribution

Type: Continuous

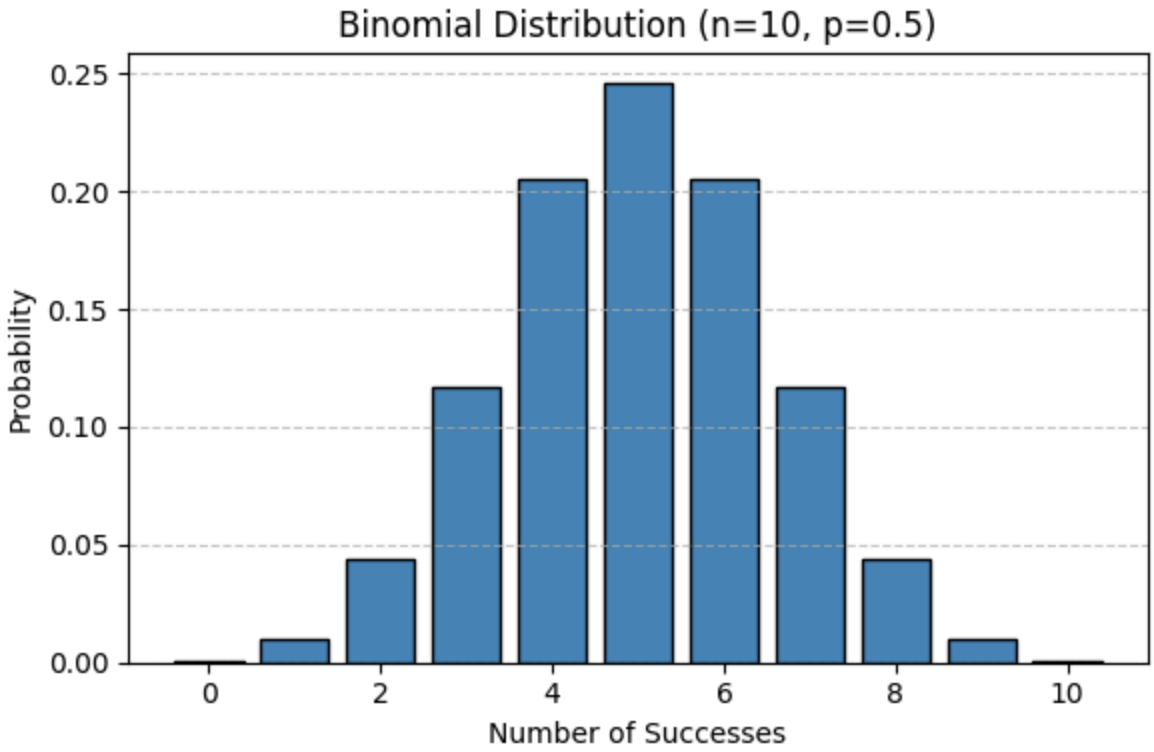
Parameters: μ (mean), σ (standard deviation) Trials: Not based on trials

Outcome: Can take any real value Shape: Symmetric, bell-shaped curve

Explanation: The Normal Distribution describes continuous data that clusters around a mean. It is widely used because many natural phenomena follow this pattern.

Example: Heights of people

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import binom, poisson, norm
import warnings
warnings.filterwarnings('ignore')

# ------------------------------------------------
# 1. BINOMIAL DISTRIBUTION
# ------------------------------------------------

n = 10       # number of trials
p = 0.5      # probability of success

x_binom = np.arange(0, n+1)
binom_pmf = binom.pmf(x_binom, n, p)

plt.figure(figsize=(6, 4))
plt.bar(x_binom, binom_pmf, color='steelblue', edgecolor='black')
plt.title('Binomial Distribution (n=10, p=0.5)')
plt.xlabel('Number of Successes')
plt.ylabel('Probability')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

# Mean and Variance
print("=== Binomial Distribution ===")
print(f"Mean     : {binom.mean(n, p)}")
print(f"Variance : {binom.var(n, p)}")
print(f"Skewness : {binom.stats(n, p, moments='s')}")
```




```
=== Binomial Distribution ===
Mean     : 5.0
Variance : 2.5
Skewness : 0.0
```

2.Real-world significance of the Poisson Distribution

The Poisson distribution is used to model the number of times an event occurs in a fixed interval when the events are rare, random, and independent. Real-world examples:

Number of customers arriving at a bank per hour

Number of emails received per day

Number of accidents at a traffic signal per week

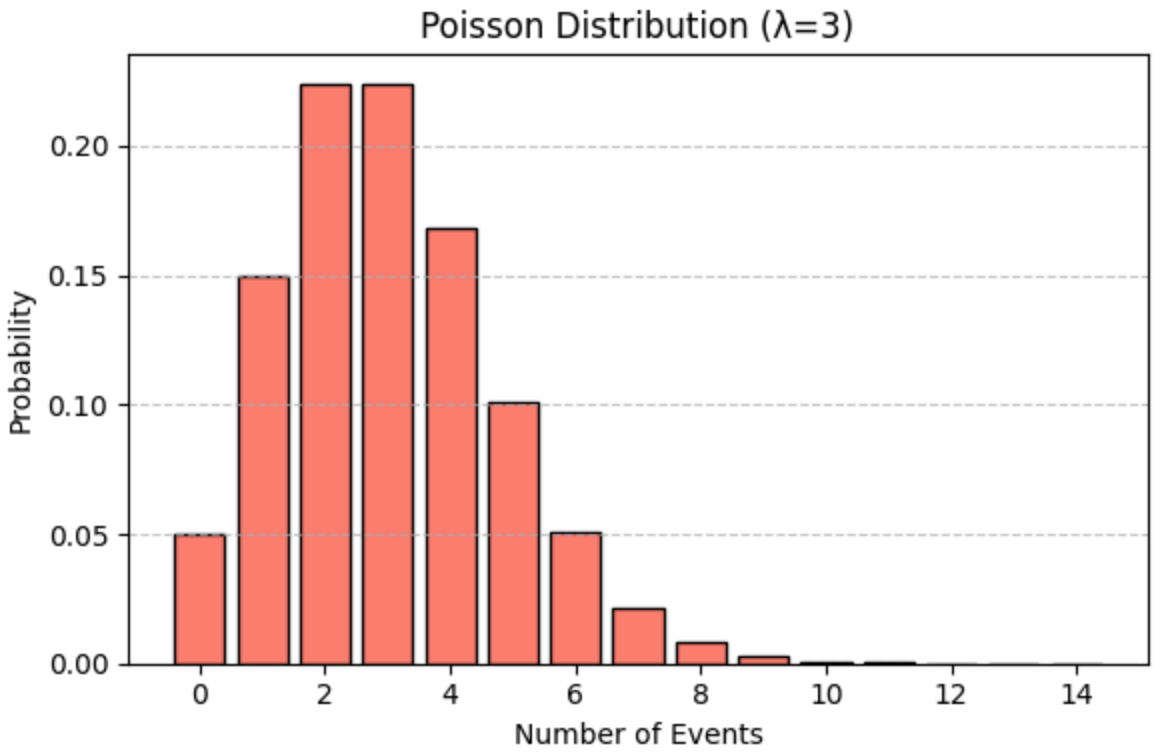
Number of defects in a manufactured product

Number of calls received at a call center per minute

It is especially useful in queuing theory, telecommunications, and reliability engineering where we need to predict how often rare events happen over time.

```
# ------------------------------------------------
# 2. POISSON DISTRIBUTION
# ------------------------------------------------

lam = 3      # average rate (lambda)

x_poisson = np.arange(0, 15)
```

```
poisson_pmf = poisson.pmf(x_poisson, lam)

plt.figure(figsize=(6, 4))
plt.bar(x_poisson, poisson_pmf, color='salmon', edgecolor='black')
plt.title('Poisson Distribution (λ=3)')
plt.xlabel('Number of Events')
plt.ylabel('Probability')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

print("\n=== Poisson Distribution ===")
print(f"Mean     : {poisson.mean(lam)}")
print(f"Variance : {poisson.var(lam)}")
print(f"Skewness : {poisson.stats(lam, moments='s')}")
```



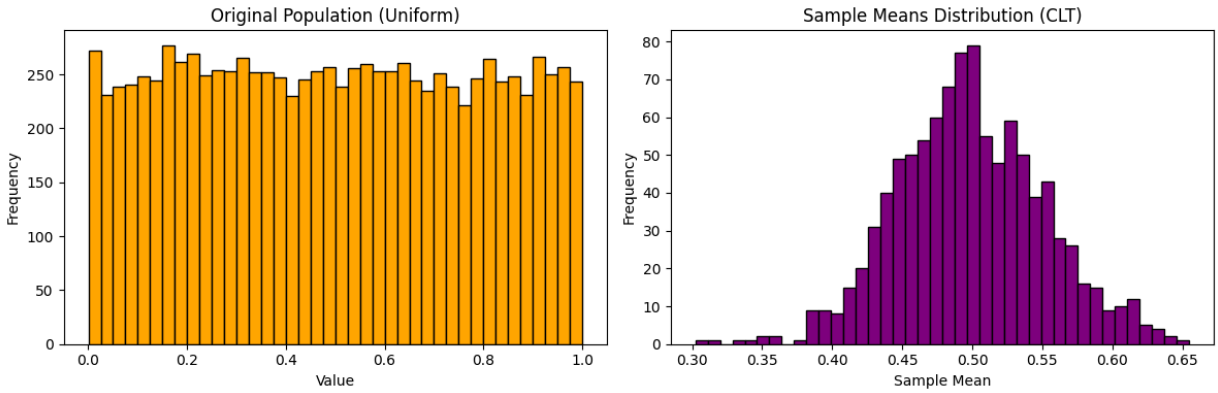

```
=== Poisson Distribution ===
Mean     : 3.0
Variance : 3.0
Skewness : 0.5773502691896257
```

3.Central Limit Theorem (CLT) and its relation to Normal Distribution

The Central Limit Theorem (CLT) states that when we take sufficiently large random samples from any population—regardless of its original distribution—the distribution of the sample means tends to approach a Normal distribution as the sample size increases, typically when n ≥ 30. This means that even if the population is uniform, skewed, or irregular, the averages of repeated samples will form a bell-shaped curve. An important

property of the CLT is that the mean of the sample means is equal to the population mean (μ), and the standard deviation of the sample means, known as the standard error, is given by σ/√n. The CLT is closely related to the Normal distribution because it explains why the Normal distribution appears so frequently in real-world data. Even when individual observations are not normally distributed, their averages tend to follow a Normal pattern, which is why the Normal distribution becomes the basis for many statistical methods such as hypothesis testing, confidence intervals, and general statistical inference.

```
# ------------------------------------------------
# 3. CENTRAL LIMIT THEOREM DEMONSTRATION
# ------------------------------------------------

# Population: Uniform distribution (not normal)
population = np.random.uniform(0, 1, 10000)
sample_means = [np.mean(np.random.choice(population, 30)) for _ in range(1000)]

plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.hist(population, bins=40, color='orange', edgecolor='black')
plt.title('Original Population (Uniform)')
plt.xlabel('Value')
plt.ylabel('Frequency')

plt.subplot(1, 2, 2)
plt.hist(sample_means, bins=40, color='purple', edgecolor='black')
plt.title('Sample Means Distribution (CLT)')
plt.xlabel('Sample Mean')
plt.ylabel('Frequency')

plt.tight_layout()
plt.show()

print("\n=== Central Limit Theorem ===")
print(f"Population Mean : {np.mean(population):.4f}")
print(f"Mean of Sample Means : {np.mean(sample_means):.4f}")
print(f"Population Std : {np.std(population):.4f}")
print(f"Std of Sample Means (Expected = σ/√n) : {np.std(sample_means):.4f}")
```



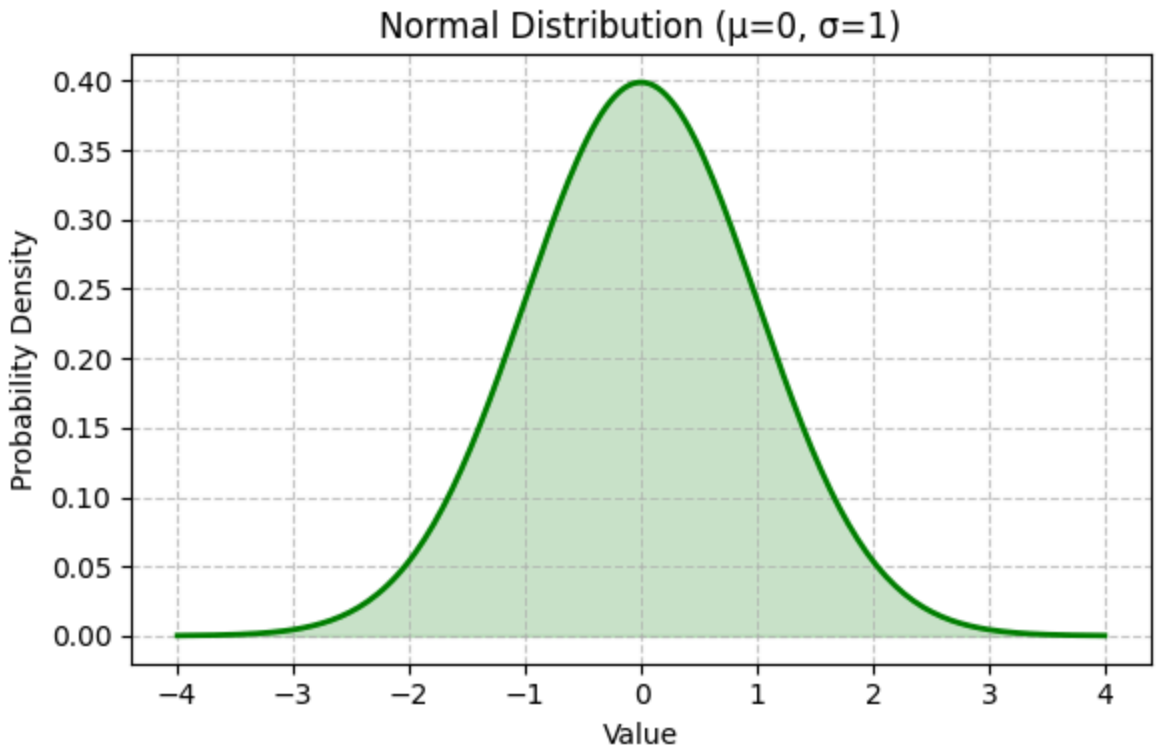

```
=== Central Limit Theorem ===
Population Mean : 0.4977
Mean of Sample Means : 0.4983
Population Std : 0.2886
Std of Sample Means (Expected = σ/√n) : 0.0525
```

4.Why Normal Distribution is widely used in Machine Learning and Statistics

The Normal Distribution is widely used in machine learning and statistics because of several important advantages. One of the main reasons is mathematical convenience, as it is completely defined by only two parameters: the mean (μ) and the standard deviation (σ). The Central Limit Theorem also supports its use, since sample means from any population tend to follow a normal distribution, which makes it useful for statistical inference. Many machine learning algorithms assume normality in the data, such as Linear Regression, Linear Discriminant Analysis (LDA), and Gaussian Naive Bayes, which simplifies modeling and computation. Another important reason is the 68-95-99.7 rule, which states that 68% of data lies within one standard deviation, 95% within two, and 99.7% within three, making interpretation very intuitive. It is also useful for detecting outliers, as extreme deviations from the mean can indicate anomalies. In addition, many real-world phenomena like heights, weights, IQ scores, and measurement errors naturally follow a normal distribution. Finally, it plays a key role in probabilistic models such as Gaussian Mixture Models (GMM) and Kalman filters, making it a fundamental concept in statistics and machine learning.

```
# ------------------------------------------------
# 4. NORMAL DISTRIBUTION
# ------------------------------------------------

mu = 0       # mean
sigma = 1    # standard deviation

x_norm = np.linspace(-4, 4, 200)
norm_pdf = norm.pdf(x_norm, mu, sigma)
```

```
plt.figure(figsize=(6, 4))
plt.plot(x_norm, norm_pdf, color='green', linewidth=2)
plt.fill_between(x_norm, norm_pdf, alpha=0.2, color='green')
plt.title('Normal Distribution (μ=0, σ=1)')
plt.xlabel('Value')
plt.ylabel('Probability Density')
plt.grid(linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

print("\n=== Normal Distribution ===")
print(f"Mean     : {norm.mean(mu, sigma)}")
print(f"Variance : {norm.var(mu, sigma)}")
print(f"Skewness : {norm.stats(mu, sigma, moments='s')}")
```




```
=== Normal Distribution ===
Mean     : 0.0
Variance : 1.0
Skewness : 0.0
```

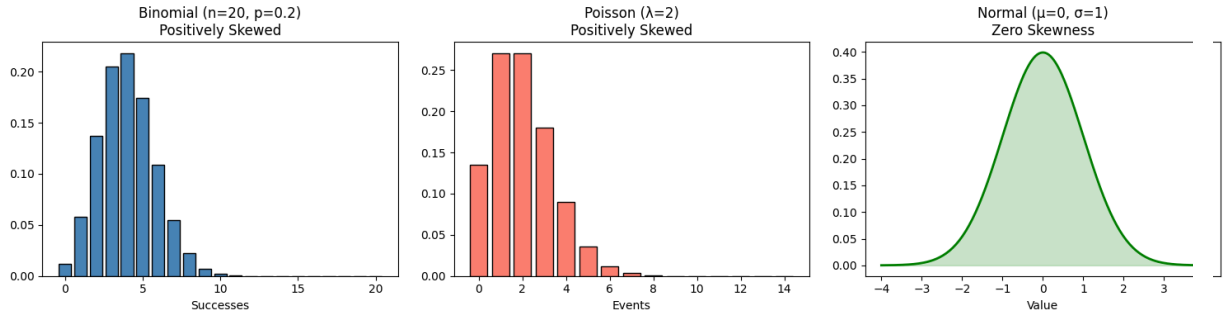
5.Skewness in distributions and which distributions exhibit it

Skewness is a statistical measure that describes the asymmetry of a distribution around its mean. When a distribution is positively skewed (right-skewed), its tail extends toward the right side, and in such cases the mean is greater than the median. In a negatively skewed (left-skewed) distribution, the tail extends toward the left side, and the mean is less than the median. A distribution with zero skewness is perfectly symmetric, meaning

the mean and median are equal. Different probability distributions exhibit different types of skewness. The Normal Distribution has zero skewness and is completely symmetric. The Binomial Distribution has skewness equal to zero when the probability of success p = 0.5; it becomes positively skewed when p < 0.5 and negatively skewed when p > 0.5. The Poisson Distribution is generally positively skewed, especially when the rate parameter λ is small, with skewness equal to 1/√λ, and it becomes more symmetric as λ increases. The Exponential Distribution is also highly positively skewed due to its rapidly decreasing probability as values increase. Similarly, the Log-Normal Distribution is strongly positively skewed because it is based on the logarithm of a normally distributed variable.

```
# ------------------------------------------------
# 5. SKEWNESS COMPARISON
# ------------------------------------------------

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# Binomial (p=0.2, right skewed)
x1 = np.arange(0, 21)
axes[0].bar(x1, binom.pmf(x1, 20, 0.2), color='steelblue', edgecolor='black')
axes[0].set_title('Binomial (n=20, p=0.2)\nPositively Skewed')
axes[0].set_xlabel('Successes')

# Poisson (lambda=2, right skewed)
x2 = np.arange(0, 15)
axes[1].bar(x2, poisson.pmf(x2, 2), color='salmon', edgecolor='black')
axes[1].set_title('Poisson (λ=2)\nPositively Skewed')
axes[1].set_xlabel('Events')

# Normal (symmetric)
x3 = np.linspace(-4, 4, 200)
axes[2].plot(x3, norm.pdf(x3, 0, 1), color='green', linewidth=2)
axes[2].fill_between(x3, norm.pdf(x3, 0, 1), alpha=0.2, color='green')
axes[2].set_title('Normal (μ=0, σ=1)\nZero Skewness')
axes[2].set_xlabel('Value')

plt.tight_layout()
plt.show()

from scipy.stats import skew

binom_data = np.random.binomial(20, 0.2, 1000)
poisson_data = np.random.poisson(2, 1000)
normal_data = np.random.normal(0, 1, 1000)

print("\n=== Skewness Comparison ===")
print(f"Binomial (p=0.2) Skewness : {skew(binom_data):.4f}")
```

```
print(f"Poisson  (λ=2)   Skewness : {skew(poisson_data):.4f}")
```




```
=== Skewness Comparison ===
Binomial (p=0.2) Skewness : 0.2578
Poisson  (λ=2)   Skewness : 0.7687
Normal   (μ=0)   Skewness : 0.0053
```

CONCLUSION

This experiment demonstrated the implementation and analysis of three fundamental probability distributions in Python. The Binomial distribution models discrete outcomes over fixed trials, the Poisson distribution models rare event counts over intervals, and the Normal distribution describes continuous data with a symmetric bell curve. The Central Limit Theorem was verified by showing that sample means from a Uniform population converge to a Normal distribution. Skewness analysis confirmed that Poisson and Binomial (with low p) are right-skewed, while Normal distribution is perfectly symmetric. These distributions form the statistical foundation for machine learning, data analysis, and inferential statistics.